

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 7. String and its Methods

Lecture: 71. String Operations

Section 1: Lecture Summary

This lecture covers several fundamental operations on Python strings beyond indexing and slicing. It revises previously taught content on arithmetic operations applied to strings, focusing on concatenation, repetition, membership testing, and string comparison. The instructor demonstrates how to concatenate strings using the `+` operator, repeat strings using ``, and check membership with `in` and `not in`. A significant part of the lecture is dedicated to string comparison using relational operators (`<`, `<=`, `>`, `>=`, `==`, `!=`). The comparison is explained based on lexicographic (dictionary) order and ASCII value ordering, clarifying how Python handles upper and lower case letters, numbers, and string length differences. Practical coding examples are shown in Jupyter Notebook to reinforce understanding.

Section 2: Key Concepts and Explanations

- String Concatenation

- Combines two strings end-to-end to form a new string.
- Uses the `+` operator.
- The original strings remain unchanged since strings are immutable in Python.

- String Repetition

- Creates a new string by repeating the original string multiple times.
- Uses the `` operator with an integer multiplier.
- Example: `x` 10 equals `xxxxxxxxxx`.
- Can repeat single letters, substrings, or complete strings.

- Membership Operations

- Checks if a substring or character exists within another string.
- Uses ``in`` for presence check (returns ``True`` or ``False``).
- Uses ``not in`` to check absence.
- Works with substrings of any length.

- String Comparison

- Performed with relational operators: ``<``, ``<=``, ``>``, ``>=``, ``==``, ``!=``.
- Based on **lexicographic order** — similar to dictionary order.
- Comparison is letter-by-letter from the start until a mismatch is found.
- If one string is a prefix of another, the shorter string is considered smaller.
- Case sensitivity matters: uppercase letters are smaller than lowercase letters.
- The order is influenced by ASCII codes:
 - Numbers (``0-9``) have ASCII values 48–57, come first.
 - Uppercase letters (``A-Z``) have ASCII values 65–90, come next.
 - Lowercase letters (``a-z``) have ASCII values 97–122, come last.
- Examples:
 - ``'apple' < 'applepie`` because the shorter word comes first.
 - ``'cat' < 'catch`` due to prefix match and extra characters in the latter.
 - ``'Data' < 'data`` because uppercase ``D`` ASCII < lowercase ``d`` ASCII.

Section 3: Example Code and Use Cases

Concatenation example

```
s1 = 'WeI'  
s2 = 'come'
```

```
s3 = s1 + s2
print(s3) # Output: Welcome
```

Explanation: Joins two strings to form a new one.

Repetition example

```
s1 = 'x'
s2 = s1 * 10
print(s2) # Output: xxxxxxxxxx
```

Explanation: Repeats the string 'x' ten times.

Membership example

```
s1 = 'abcdef'
print('a' in s1)      # Output: True
print('cde' in s1)    # Output: True
print('z' in s1)      # Output: False
print('z' not in s1)  # Output: True
```

Explanation: Checks if specific characters or substrings exist in s1.

String comparison examples

```
print('apple' < 'ball')      # True (dictionary order)
print('applepie' > 'apple')  # True ('applepie' longer, after all common
                             # chars)
print('cat' < 'catch')       # True (prefix comparison)
print('Data' < 'data')       # True (uppercase < lowercase)
print('2nd' < 'Byte')        # True (numbers < uppercase letters)
print('software' > 'Hardware') # True (lowercase 's' > uppercase 'H' in
                             # ASCII)
print('python' < 'pycharm')   # False ('t' > 'c')
print('printer' > 'print')    # True (longer string with prefix match)
```

Explanation: Demonstrates how Python compares strings based on lexicographic and ASCII order.

Section 4: Key Takeaways

- Use `+` to concatenate strings, resulting in a new combined string.

- Use `` to repeat a string multiple times, creating a new string.
- Use `in` and `not in` to test for substring presence or absence.
- String comparison uses lexicographic ordering, comparing characters from left to right.
- Uppercase letters come before lowercase letters in ASCII order.
- Numbers come before uppercase letters, which come before lowercase letters.
- When comparing strings with equal prefixes, the shorter string is smaller.
- String operations do not modify original strings due to immutability of strings.
- Practice each operation in an interactive Python environment like Jupyter Notebook for better understanding.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Operations on String



Operations on String

Operations on String

Concatenation

Operations on String

Concatenation

+

Operations on String

Concatenation

+

Repetition

Operations on String

Concatenation

+

Repetition

*

Operations on String

Concatenation +

Repetition *

Membership

Operations on String

Concatenation +

Repetition *

Membership
in , not in

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python [conda env:base]

[]:

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python [conda env:base]

[]:

Operations on String

Concatenation
+

Repetition
*

Membership
in , not in

String Comparison
< <= > >= == !=

```
[ ]: s1 = 'Wel'
```

s1

W	e	l
0	1	2

Operations on String

Concatenation
+

Repetition
*

Membership
in , not in

String Comparison
< <= > >= == !=

```
[ ]: s1 = 'Wel'
s2 = 'come'
```

s1

W	e	l
0	1	2

s2

c	o	m	e
0	1	2	3

Operations on String

Concatenation
+

Repetition
*

Membership
in , not in

String Comparison
< <= > >= == !=

```
[ ]: s1 = 'Wel'
s2 = 'come'
```

s1

W	e	l
0	1	2

s2

c	o	m	e
0	1	2	3

Operations on String

Concatenation
+

Repetition
*

Membership
in , not in

String Comparison
< <= > >= == !=

```
[ ]: s1 = 'Wel'
s2 = 'come'
s3 = s1 + s2
```

s1

W	e	l
0	1	2

+

s2

c	o	m	e
0	1	2	3

Operations on String

Concatenation +

Repetition *

Membership in , not in

String Comparison < <= > >= == !=

```
[1]: s1 = 'Wel'
s2 = 'come'

s3 = s1 + s2

[ ]:
```

s1: W e l + s2: c o m e

s3: W e l c o m e (indices 0-6)

Operations on String

Concatenation +

Repetition *

Membership in , not in

String Comparison < <= > >= == !=

```
[1]: s1 = 'Wel'
s2 = 'come'

s3 = s1 + s2

[ ]: print(s3)
```

s1: W e l + s2: c o m e

s3: W e l c o m e (indices 0-6)

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base]

[1]: s1 = 'Wel'
s2 = 'come'
s3 = s1 + s2

[3]: print(s3)
Welcome

[]:

s1 Wel + s2 come

s3 Wel come
0 1 2 3 4 5 6

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base]

[]:

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help Trusted

Python [conda env:base] • JupyterLab

```
[ ]: s1 = 'x'
```

s1

x

0

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

```
[ ]: s1 = 'x'
      s2 = s1 * 10
```

s1 x * 10

0

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

```
[5]: s1 = 'x'
      s2 = s1 * 10
```

```
[ ]:
```

s1 x * 10

0

s2 x x x x x x x x x x

0 1 2 3 4 5 6 7 8 9

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

```
[5]: s1 = 'x'
s2 = s1 * 10

[7]: print(s2)

xxxxxxx
```

s1

x

* 10

0

s2

x x x x x x x x x x

0 1 2 3 4 5 6 7 8 9

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

```
[ ]:
```

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base] * ○ ≡

[]: s1 = 'abcdef'

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base] * ○ ≡

[9]: s1 = 'abcdef'

[]:

s1

a	b	c	d	e	f
0	1	2	3	4	5

Operations on String

Concatenation
+

Repetition
*

Membership
in , not in

String Comparison
< <= > >= == !=

```
[9]: s1 = 'abcdef'
[ ]: 'a' in s1
```

s1 a b c d e f
0 1 2 3 4 5

Operations on String

Concatenation
+

Repetition
*

Membership
in , not in

String Comparison
< <= > >= == !=

```
[9]: s1 = 'abcdef'
[11]: 'a' in s1
[11]: True
[ ]:
```

s1 a b c d e f
0 1 2 3 4 5

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python [conda env:base]

[9]: s1 = 'abcdef'

[11]: 'a' in s1

[11]: True

[13]: 'cde' in s1

s1

a	b	c	d	e	f
0	1	2	3	4	5

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python [conda env:base]

[9]: s1 = 'abcdef'

[11]: 'a' in s1

[11]: True

[13]: 'cde' in s1

[13]: True

[15]:

s1

a	b	c	d	e	f
0	1	2	3	4	5

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

```
localhost
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python [conda env:base]

[9]: s1 = 'abcdef'
[11]: 'a' in s1
[11]: True
[13]: 'cde' in s1
[13]: True
[ ]: 'z' in s1
```

s1

a	b	c	d	e	f
0	1	2	3	4	5

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

localhost

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python [conda env:base] * ○ □

[9]: s1 = 'abcdef'

[11]: 'a' in s1

[11]: True

[13]: 'cde' in s1

[13]: True

[15]: 'z' in s1

[15]: False

[]:

s1

a b c d e f

0 1 2 3 4 5

Operations on String

Concatenation

+

Repetition

*

Membership

in , not in

String Comparison

< <= > >= == !=

String Comparison

< <= > >= == !=

String Comparison

< <= > >= == !=

Lexicographic Order

String Comparison

< <= > >= == !=

Lexicographic Order

Dictionary Order

String Comparison

< <= > >= == !=

Dictionary Order

String Comparison

< <= > >= == !=

Dictionary Order

apple

String Comparison

< <= > >= == !=

Dictionary Order

apple

ball

String Comparison

< <= > >= == !=

Dictionary Order

apple

ball

cat

String Comparison

< <= > >= == !=

Dictionary Order

apple

ball

cat

dog

String Comparison

< <= > >= == !=

Dictionary Order

apple

ball

cat

dog

python

String Comparison

< <= > >= == !=

Dictionary Order

apple

ball

cat

dog

python

apple

String Comparison

< <= > >= == !=

Dictionary Order

apple

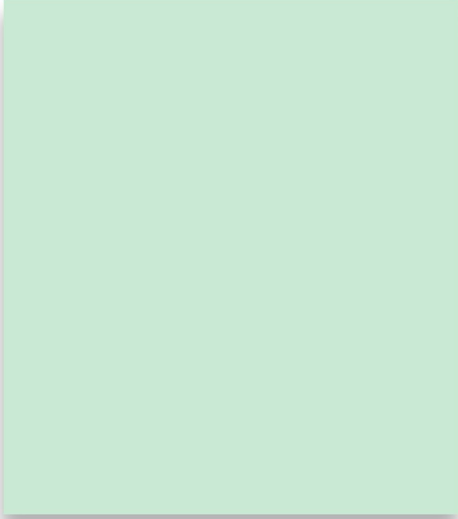
ball

cat

dog

python

apple < ball



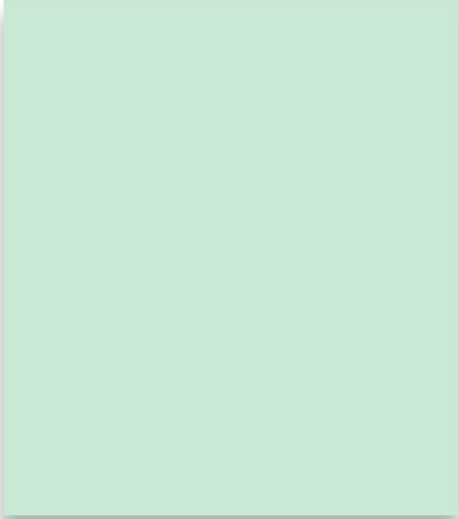
String Comparison

< <= > >= == !=

Dictionary Order

apple
ball
cat
dog
python

apple < ball < cat



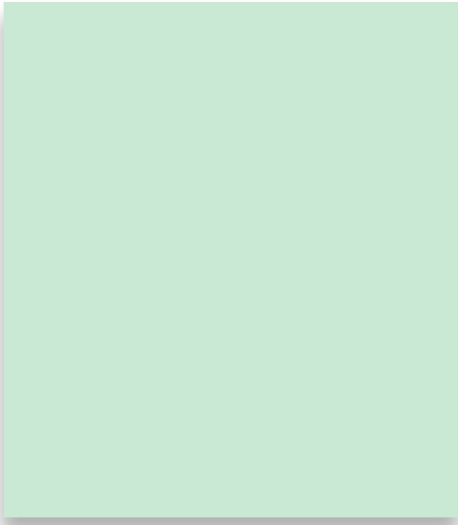
String Comparison

< <= > >= == !=

Dictionary Order

apple
ball
cat
dog
python

apple < ball < cat < dog



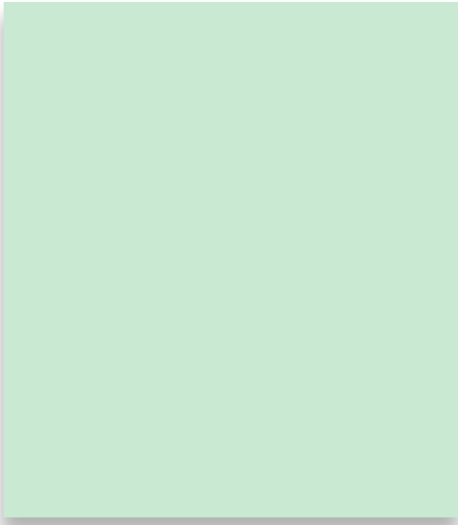
String Comparison

< <= > >= == !=

Dictionary Order

apple
ball
cat
dog
python

apple < ball < cat < dog < python

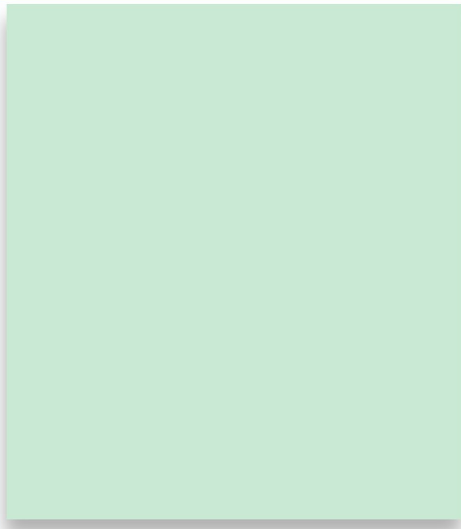


String Comparison

< <= > >= == !=

Dictionary Order

apply > apple



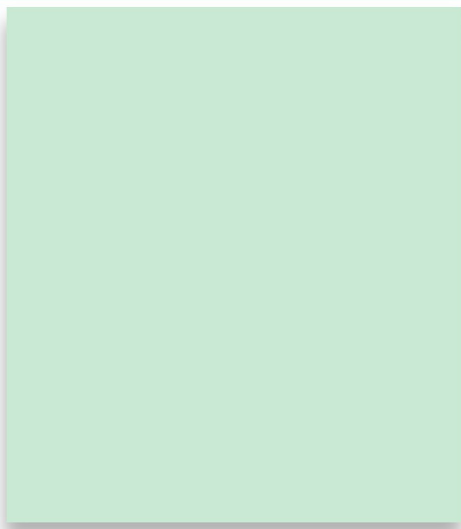
String Comparison

< <= > >= == !=

Dictionary Order

apply > apple

cat < catch



String Comparison

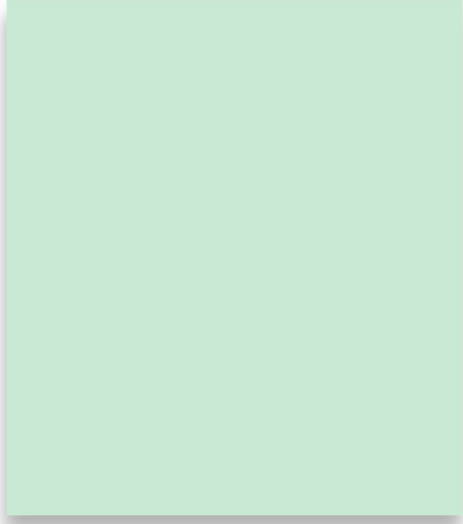
< <= > >= == !=

Dictionary Order

apply > apple

cat < catch

data > Data



String Comparison

< <= > >= == !=

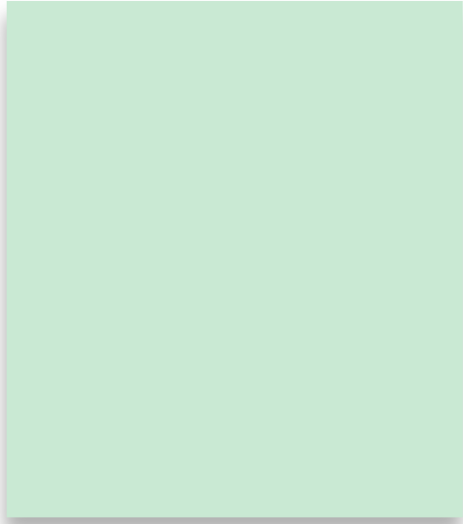
Dictionary Order

apply > apple

cat < catch

data > Data

2nd < Byte



String Comparison

< <= > >= == !=

Dictionary Order

0 < 1 < 2 < 3....

String Comparison

< <= > >= == !=

Dictionary Order

0 < 1 < 2 < 3.....< A < B < C

String Comparison

< <= > >= == !=

Dictionary Order

0 < 1 < 2 < 3.....< A < B < C< a < b < c ...

String Comparison

< <= > >= == !=

Dictionary Order

0 < 1 < 2 < 3.....< A < B < C< a < b < c ...

48 - 57

String Comparison

< <= > >= == !=

Dictionary Order

0 < 1 < 2 < 3.....< A < B < C< a < b < c ...

48 - 57 65 - 90

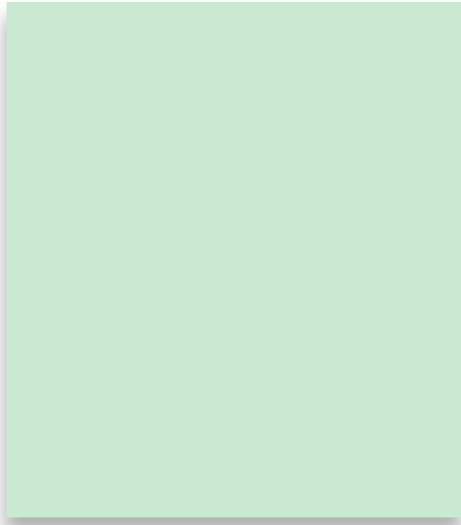
String Comparison

< <= > >= == !=

Dictionary Order

0 < 1 < 2 < 3.....< A < B < C< a < b < c ...

48 - 57 65 - 90 97 - 122

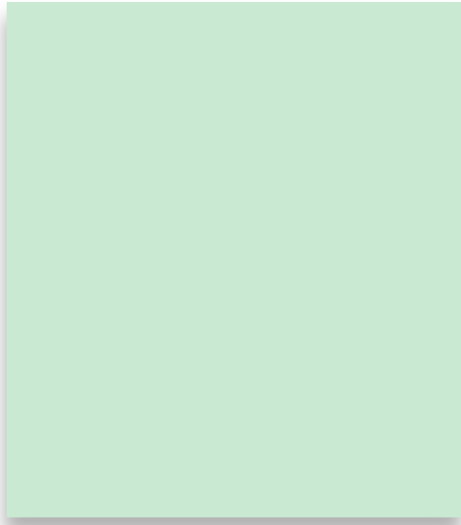


String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware'

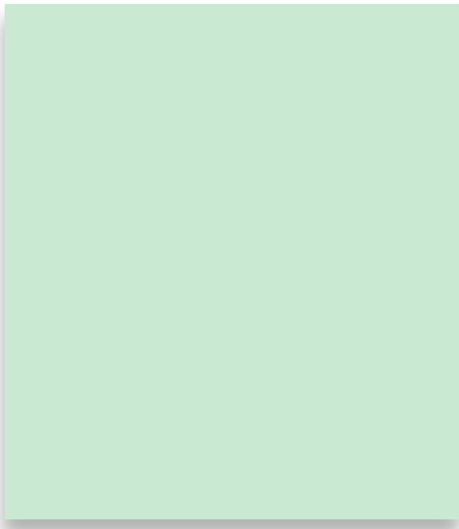


String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware' s1 > s2

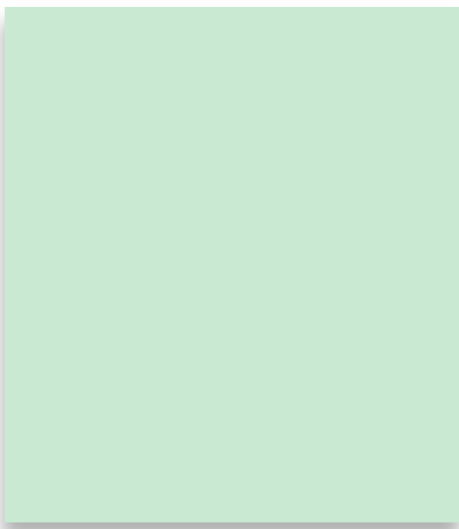


String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware' True ← s1 > s2



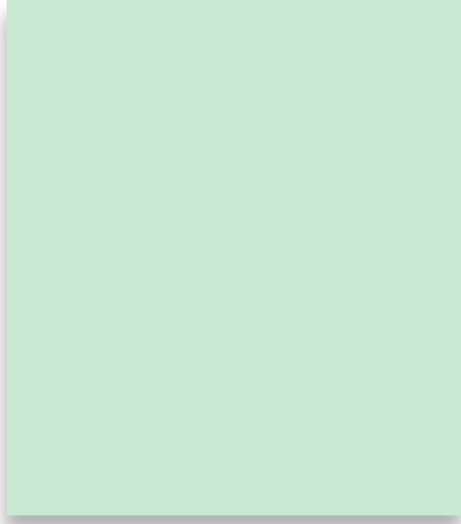
String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware' True ← s1 > s2

s1 = 'python'
s2 = 'pycharm'



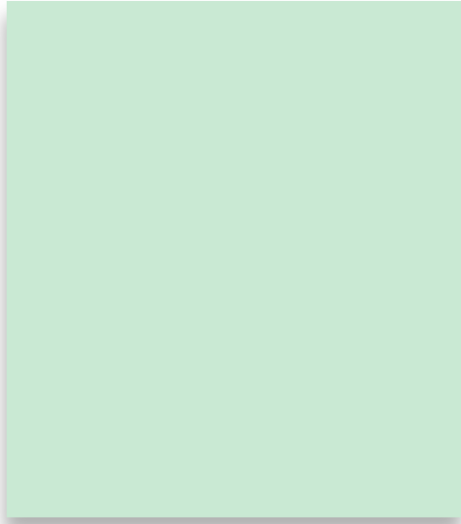
String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware' True — s1 > s2

s1 = 'python'
s2 = 'pycharm' s1 < s2



String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware' True — s1 > s2

s1 = 'python'
s2 = 'pycharm' False — s1 < s2

String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware' True ← s1 > s2

s1 = 'python'
s2 = 'pycharm' False ← s1 < s2

s1 = 'integer'
s2 = 'integer'

String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'

s2 = 'Hardware'

True ← s1 > s2

s1 = 'python'

s2 = 'pycharm'

False ← s1 < s2

s1 = 'integer'

s2 = 'integer'

s1 == s2

String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'

s2 = 'Hardware'

True ← s1 > s2

s1 = 'python'

s2 = 'pycharm'

False ← s1 < s2

s1 = 'integer'

s2 = 'integer'

True ← s1 == s2

String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'

s2 = 'Hardware'

True — s1 > s2

s1 = 'python'

s2 = 'pycharm'

False — s1 < s2

s1 = 'integer'

s2 = 'integer'

True — s1 == s2

s1 = 'printer'

s2 = 'print'

String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'

s2 = 'Hardware'

True — s1 > s2

s1 = 'python'

s2 = 'pycharm'

False — s1 < s2

s1 = 'integer'

s2 = 'integer'

True — s1 == s2

s1 = 'printer'

s2 = 'print'

s1 > s2

String Comparison

< <= > >= == !=

Dictionary Order

s1 = 'software'
s2 = 'Hardware' True ← s1 > s2

s1 = 'python'
s2 = 'pycharm' False ← s1 < s2

s1 = 'integer'
s2 = 'integer' True ← s1 == s2

s1 = 'printer'
s2 = 'print' True ← s1 > s2